Notes Langage C

EIC95

Commentaires

Les commentaires sur une seule ligne commencent par deux barres obliques (**//**) et les commentaires sur plusieurs lignes commencent par **/*** et se terminent par ***/**.

```
int main()
{
    //azertyuiop
    /*qwerty
    uiop*/
}
```

Variables

Pour créer une variable, spécifiez le type et attribuez-lui une valeur.

ATTENTION :On ne peut pas appeler une variable n'importe comment :

- Il ne peut y avoir que des minuscules, des majuscules et des chiffres;
- Votre nom de variabledoit commencer par une lettre;
- Les espaces sont interdits mais on peut utiliser le caractère "underscore" _ pour séparer des mots. C'est le seul caractère différent des lettreset chiffres autorisé;
- Les accents ("é", "à", "ê", etc) et les symboles(+, -, *, /, %, =) sont interdits.

int	2 ou 4 octets	Stocke des nombres entiers, sans décimales
float	4 octets	Stocke des nombres fractionnaires, contenant une ou plusieurs décimales. Suffisant pour stocker 6-7 chiffres décimaux
double	8 octets	Stocke des nombres fractionnaires, contenant une ou plusieurs décimales. Suffisant pour stocker 15 chiffres décimaux
char	1 octet	Stocke un seul caractère/lettre/chiffre, ou des valeurs ASCII

```c
int main()
{
    int a = 5;
    char letter = 'A';
    return 0;
}
```

Conversion de types

Il existe deux types de conversion en C :

- Conversion implicite (automatique): La conversion implicite est effectuée automatiquement par le compilateur lorsque vous attribuez une valeur d'un type à un autre.
- Conversion explicite (manuellement): La conversion explicite est effectuée manuellement en plaçant le type entre parenthèses () devant la valeur.

```
int main()
{
    int nbr = 5;
    int nbr_2 = (float) nbr; //explicit
    char letter = nbr; //implicit
}
```

Constantes

Pour déclarer une constante, vous pouvez utiliser le mot-clé **const**.

```
int main()
{
    const nombre = 124;
}
```

Ecriture

Pour afficher des valeurs ou imprimer du texte en C, vous pouvez utiliser la fonction **printf()**. Pour insérer une nouvelle ligne, vous pouvez utiliser le caractère **\n**. Pour afficher des variables, utilisez différents spécificateurs de format pour chaque type de données.

%d ou %i	int
%f ou %F	float
%lf	double
%c	char
%s	Chaînes de caractères (texte)

```c
int main()
{
    int nbr = 5;
    printf("HELLO WORLD\n");
    printf("%d" , nbr);
}
```

\t	Crée un onglet horizontal
****	Insère un caractère barre oblique inverse (\)
\"	Insère un caractère de guillemet double (")
\n	Insère un saut de ligne (retour à la ligne)
\r	Insère un retour chariot (début de ligne)
\b	Insère une suppression (backspace)

Entrées utilisateur

Pour obtenir la saisie de l'utilisateur , vous pouvez utiliser la fonction **scanf()**.

La fonction scanf() prend deux arguments : le spécificateur de format de la variable (**%d** par exemple) et l'opérateur de référence (**&myNum** par exemple), qui stocke l'adresse mémoire de la variable.

```
int main()
{
    int nbr;
    scanf("%d" , &nbr);
}
```

La fonction **fgets** lit une ligne de texte à partir d'un flux de fichier et la stocke dans un tampon. Elle lit jusqu'à n-1 caractères ou jusqu'à ce qu'une nouvelle ligne soit rencontrée, en ajoutant un caractère nul ('**\0**') à la fin de la chaîne. Si le tampon est suffisamment grand, elle lit toute la ligne, sinon elle se limite à n-1 caractères pour éviter les débordements. La fonction retourne un pointeur vers le tampon en cas de succès, ou NULL si une erreur survient ou si la fin du fichier est atteinte avant la lecture.

```c
#include <stdio.h>

int main() {
    char line[50];

    if (fgets(line, sizeof(line), stdin)) {
        printf("Input : %s", line);
    }

    return 0;
}
```

Opérateurs

Arithmétiques

+	Addition
-	Soustraction
*****	Multiplication
/	Division
%	Modulo (reste de la division entière)

Raccourcis

+=	Addition (x += y équivaut à x = x + y)
-=	Soustraction (x -= y équivaut à x = x - y)
***=**	Multiplication (x *= y équivaut à x = x * y)
/=	Division (x /= y équivaut à x = x / y)
%=	Modulo (x %= y équivaut à x = x % y)
++	Incrémentation
--	Décrémentation

Comparaison

==	Égalité
!=	Inégalité
<	Inférieur à
>	Supérieur à
<=	Inférieur ou égal à
>=	Supérieur ou égal à

Logique

&&	ET logique
||	OU logique
!	NON logique

Mathématiques

Voici une liste des fonctions les plus utiles de la bibliothèque **<math.h>** en C.

sqrt(x)	Retourne la racine carrée de x.
pow(base, exp)	Retourne base élevé à la puissance exp.
fabs(x)	Retourne la valeur absolue de x.
ceil(x)	Retourne le plus petit entier supérieur ou égal à x.
floor(x)	Retourne le plus grand entier inférieur ou égal à x.

exp(x)	Retourne e élevé à la puissance x, où e est la base des logarithmes naturels.
log(x)	Retourne le logarithme naturel (base e) de x.
log10(x)	Retourne le logarithme en base 10 de x.
sin(x)	Retourne le sinus de x, où x est en radians.
cos(x)	Retourne le cosinus de x, où x est en radians.
tan(x)	Retourne la tangente de x, où x est en radians.

Conditions

if

```c
int main()
{
    if (20 > 18) {
        printf("20 is greater than 18");
    }
}
```

if..else

```c
int main()
{
    int time = 20;
    if (time < 18) {
        printf("Good day.");
    } else {
        printf("Good evening.");
    }
}
```

else if

```c
int main()
{
    int time = 22;
    if (time < 10) {
        printf("Good morning.");
    } else if (time < 20) {
        printf("Good day.");
    } else {
        printf("Good evening.");
    }
}
```

switch case

```c
int main()
{
    int day = 4;
    switch (day) {
        case 1:
            printf("Monday");
            break;
        case 2:
            printf("Tuesday");
            break;
        case 3:
            printf("Wednesday");
            break;
        case 4:
            printf("Thursday");
            break;
        case 5:
            printf("Friday");
            break;
        case 6:
            printf("Saturday");
            break;
        case 7:
            printf("Sunday");
            break;
    }

}
```

Ternaire

```c
int main()
{
    int time = 20;
    (time < 18) ? printf("Good day.") : printf("Good evening.");
}
```

Boucles

while

```c
int main()
{
    int i = 0;
    while (i < 5) {
        printf("%d\n", i);
        i++;
    }
}
```

do...while

```c
int main()
{
    int i = 0;
    do {
        printf("%d\n", i);
        i++;
    }while (i < 5);
}
```

for

```c
int main()
{
    for (int i = 0; i < 5; i++) {
        printf("%d\n", i);
    }
}
```

Tableaux

Pour créer un tableau, définissez le type de données et spécifiez le nom du tableau suivi de crochets **[]**. Pour accéder à un élément de tableau, reportez-vous à son numéro d'index. Pour obtenir la taille d'un tableau, vous pouvez utiliser l'opérateur **sizeof**, par la taille d’un element du tableau.

```
int main()
{
    int tab[] = {1,2,3,4,5,6,7,8,9};
    printf("%d" , sizeof(tab)/sizeof(tab[0]));
}
```

Pour créer un tableau multidimensionnel en C, il suffit de déclarer un tableau avec plusieurs dimensions, comme si vous créiez un tableau de tableaux.

```c
int main()
{
    int tab[3][3] = {{1,2,3},{4,5,6}};
    printf("%d" , tab[0][0]);
}
```

Fonctions

Pour créer une fonction en C, définissez-la en précisant son type de retour, son nom et ses paramètres, puis fournissez le code de la fonction entre accolades. Ecrivez le nom de la fonction suivi de parenthèses dans lesquelles vous mettez les paramètres pour apeller la fonction.

```c
int main()
{
    int add(int a, int b){
        return a + b;
    }
    void print(){
        printf("HELLO WORLD");
    }
    add(5,6);
    print();
}
```

La **programmation modulaire** est une technique qui consiste à diviser un programme en sous-programmes ou modules pour accomplir des tâches spécifiques. Chaque module est conçu pour réaliser une partie de la fonctionnalité globale et est conçu de manière à être réutilisable. Les fonctions sont déclarées dans des fichiers d'en-tête (**.h**) et définies dans des fichiers sources (**.c**). Les fichiers d'en-tête sont inclus dans les fichiers sources via la directive #include.

Un **prototype de fonction** est une déclaration de la fonction qui informe le compilateur de la manière dont la fonction est utilisée. Il comprend le type de retour de la fonction, le nom de la fonction, les types d'arguments qu'elle prend et un point-virgule. Supposons que vous ayez une fonction add qui additionne deux nombres. Vous pouvez mettre le prototype de la fonction dans un fichier d'en-tête add.h, la définition de la fonction dans un fichier source add.c, et ensuite inclure add.h dans tout autre fichier qui utilise la fonction add.

add.h

```
#ifndef ADD_H
#define ADD_H

int add( int a , int b);

#endif
```

add.c

```c
#include "add.h"


int add( int a , int b){
    return a + b;
}
```

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include "add.h"

int main()
{
    printf("%d" , add(5,6));
}
```

Pour la compilation, nous ecrirons **gcc -o main.exe main.c add.c**. Les lignes **ifndef**, **define** et **endif** sont là pour empêcher le compilateur d'inclure plusieurs fois ce fichier.

static

Pour créer une variable globale accessible uniquement dans un fichier, rajoutez simplement le mot-clé **static** devant (cela marche aussi pour les fonctions).

```
int main()
{
    static int nbr = 5;
}
```

Pointeurs

Un pointeur est une variable qui stocke l'adresse mémoire d'une autre variable. Pour déclarer un pointeur, on utilise le symbole *****. Les pointeurs doivent être du même type que la variable qu'ils pointent. Un pointeur peut être nul, signifiant qu'il ne pointe vers aucune adresse valide. On utilise généralement la constante **null** pour initialiser un pointeur nul. Utilisez l'opérateur ***** pour accéder ou modifier la valeur pointée par le pointeur. Il permettent de modifier les arguments d'une fonction ou de gérer des tableaux dynamiques.

```c
void print(int* ptr){
    printf("%d" , *ptr);
}

int main()
{
    int x = 5;
    int* ptr = &x;
    print(ptr);
}
```

Chaines de caractères

Une chaîne de caractères en C est une séquence de caractères stockée dans une région de mémoire contiguë. Elle est représentée sous forme de tableaux de char appelés chaînes de caractères. Une chaîne de caractères se termine toujours par un caractère spécial appelé "caractère de fin de chaîne", représenté par **\0**. Ce caractère indique à l'ordinateur où s'arrête la chaîne. Pour stocker une chaîne de n caractères, le tableau doit avoir une taille de n + 1 pour inclure le caractère de fin de chaîne. La bibliothèque **string.h** fournit de nombreuses fonctions dédiées aux calculs sur des chaînes.

strlen(s)	Retourne la longueur de la chaîne s (sans le caractère nul).
strcpy(dest, src)	Copie la chaîne src dans dest.
strcat(dest, src)	Concatène la chaîne src à la fin de dest.
strcmp(s1, s2)	Compare les chaînes s1 et s2.
strchr(s, c)	Retourne un pointeur vers la première occurrence du caractère c dans s.
strstr(haystack, needle)	Retourne un pointeur vers la première occurrence de la sous-chaîne needle dans haystack.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char hello[] = {'H','e','l','l','o','\0'};
    char world[] = "World"; //only for initialisation
    printf("%s %s\n" , hello , world);
    printf("%d" , strlen(hello));
}
```

Directives de préprocesseurs

Les directives de préprocesseur en C sont des commandes qui sont traitées avant la compilation du code. Elles commencent généralement par le symbole # et permettent de contrôler le processus de compilation :

- **#include** : Inclut le contenu d'un fichier.
- **#define** : Définit une macro.
- **#undef** : Annule la définition d'une macro.
- **#ifdef** : Vérifie si une macro est définie.

#ifndef : Vérifie si une macro n'est pas définie.

#else : Fournit une alternative dans une conditionnelle.

#endif : Termine une section conditionnelle.

#if : Évalue une expression conditionnelle.

#error : Génère un message d'erreur.

#pragma : Donne des instructions spécifiques au compilateur.

```c
#include <stdio.h>
#include <stdlib.h>
#define nbr 5

int main()
{
    int tab[nbr] = {1,2,3,4,5};
}
```

Le préprocesseur offre des constantes prédéfinies telles que **_LINE_** , **_FILE_** , **_DATE_** , et **_TIME_** . Ces constantes peuvent être utilisées pour afficher des informations telles que le numéro de ligne, le nom du fichier, la date et l'heure de compilation.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("%s" , __DATE__);
}
```

Structures

Les structures (également appelées structs) sont un moyen de regrouper plusieurs variables liées en un seul endroit. Chaque variable de la structure est appelée membre de la structure. Contrairement à un tableau , une structure peut contenir de nombreux types de données différents (int, float, char, etc.). Vous pouvez créer une structure en utilisant le mot-clé **struct** et déclarer chacun de ses membres entre accolades. Pour accéder aux membres d'une structure, utilisez la syntaxe à points (**.**).

```c
struct myStruct {
    int a;
    char letter;
};

int main()
{
    struct myStruct var;
    var.letter = 'A';
}
```

Enumérations

Pour créer une énumération, utilisez le mot-clé **enum**, suivi du nom de l'énumération, et séparez les éléments de l'énumération par une virgule. Pour accéder à l'énumération, vous devez créer une variable de celle-ci. À l'intérieur de la méthode main(), spécifiez le mot-clé enum, suivi du nom de l'énumération. Par défaut, le premier élément a la valeur 0, le deuxième a la valeur 1, etc. Pour donner plus de sens aux valeurs, vous pouvez facilement les modifier, Mais notez que si vous attribuez une valeur à un élément spécifique, les éléments suivants mettront à jour leurs numéros en conséquence.

```
enum Level {
    Low = 25,
    Medium = 50,
    High // 51
};

int main()
{
    enum Level var = Low;
}
```

Gestion de fichiers

La gestion de fichiers en C consiste à ouvrir, lire, écrire et fermer des fichiers. Voici un résumé des opérations principales :

- Utilisez **fopen(nom_fichier, mode)** pour ouvrir un fichier dans un mode spécifique (lecture, écriture, etc.). Retourne un pointeur vers le fichier ou NULL en cas d'erreur.
- Utilisez **fread(buffer, taille, nombre, fichier)** pour lire des données depuis un fichier binaire, ou **fgets(buffer, taille, fichier)** pour lire une ligne de texte.

- Utilisez **fwrite(buffer, taille, nombre, fichier)** pour écrire des données dans un fichier binaire, ou **fprintf(fichier, format, ...)** pour écrire du texte formaté.
- Utilisez **fclose(fichier)** pour fermer un fichier ouvert, libérant ainsi les ressources associées.
- Utilisez **fseek(fichier, offset, origine)** pour déplacer le curseur de lecture/écriture à une position spécifique. **ftell(fichier)** retourne la position actuelle du curseur.
- Utilisez **feof(fichier)** pour vérifier la fin du fichier, **ferror(fichier)** pour détecter les erreurs de lecture/écriture, et **clearerr(fichier)** pour réinitialiser les indicateurs d'erreur.

Mode	Description
"r"	Lecture seule. Ouvre le fichier pour lecture. Le fichier doit exister.
"w"	Écriture seule. Crée un nouveau fichier ou écrase un fichier existant.
"a"	Ajout seule. Ouvre le fichier pour écriture à la fin du fichier. Crée le fichier s'il n'existe pas.

"r+"	Lecture et écriture. Ouvre le fichier pour lecture et écriture. Le fichier doit exister.
"w+"	Lecture et écriture. Crée un nouveau fichier ou écrase un fichier existant, et permet la lecture et l'écriture.
"a+"	Lecture et écriture. Ouvre le fichier pour lecture et écriture à la fin du fichier. Crée le fichier s'il n'existe pas.

```c
#include <stdio.h>

int main() {
    FILE *file = fopen("example.txt", "w");
    if (file == NULL) return 1;
    fprintf(file, "Hello, world !\n");
    fclose(file);

    file = fopen("exemple.txt", "r");
    if (file == NULL) return 1;
    char line[100];
    while (fgets(line, sizeof(line), file)) {
        printf("%s", line);
    }
    fclose(file);

    return 0;
}
```

Allocation dynamique

L'allocation dynamique en C permet de gérer la mémoire de manière flexible pendant l'exécution du programme. Voici un résumé de ses fonctions principales :

- **malloc(size_t size)** : Alloue un bloc de mémoire de size octets et retourne un pointeur vers le début de ce bloc. La mémoire n'est pas initialisée.
- **calloc(size_t num, size_t size)** : Alloue de la mémoire pour un tableau de num éléments, chacun de size octets. La mémoire est initialisée à zéro.

- **realloc(void *ptr, size_t size)** : Réalloue un bloc de mémoire précédemment alloué à ptr pour avoir une nouvelle taille de size octets. Peut déplacer le bloc à un nouvel emplacement.
- **free(void *ptr)** : Libère un bloc de mémoire précédemment alloué. ptr ne doit plus être utilisé après free.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = malloc(sizeof(int));
    if (ptr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    *ptr = 42;
    printf("Value: %d\n", *ptr);

    free(ptr);

    return 0;
}
```

FIN